technische universität
dortmund

Case 1:
Machine Learning Applications

IT Management

Winter Term 2026/27

Submit you solution until **02.11.2026 midnight to the moodle system**. Please provide **all involved names and matriculation numbers** (max. 3) on the front page.

Please submit as one **PDF**. Please prepare a PDF for the discussion on Tuesday after submission using the guiding questions (about 5 slides).

Technische Universität Dortmund

Lehrstuhl für Enterprise Computing
Prof. Dr. Christian Janiesch
Fakultät für Informatik

Machine Learning Applications

1 TensorFlow Playground

1.1 Introduction

With this short exercise, you **interactively observe**:

(a) why a linear boundary is insufficient for a ring,
(b) how a hidden layer with a nonlinear activation bends the decision curve,
(c) what role the learning rate plays for stability and speed, why L2 tends to smooth and help under noise.

During this exercise, you will encounter several technical terms from the machine learning domain. Please see the glossary (Section 3) in case of doubt.

Open the TensorFlow Playground in your browser (https://playground.tensorflow.org). Your goal is to **observe and explain** what happens when you change a few simple settings in an interactive machine-learning demo. No programming is required.

Begin with this basic setup: choose **Problem** = **Classification**, **Dataset** = **Circle**, and switch **Show test data** to **On** so that held-out points appear. Under **Features**, turn **on only x1** and **x2** and leave all other features off; this ensures the model learns only from these two inputs. Set the **Network** to **0 hidden layers** (no hidden layer), the **Learning rate** to **0.03**, and **Regularization** to **None** (rate 0). Press Play. On the canvas you will see a **decision boundary**, the line or curve the model uses to separate blue from orange points.

1.2 Capacity and Non-linearity

Now consider why this first attempt struggles. With **0 hidden layers**, the model is effectively **linear**; its decision boundary is **a straight line**. The **Circle** dataset forms a **ring**: points in the inner disk belong to one class, points in the outer ring to the other. A single straight line **cannot cleanly separate** inner from outer points; it must slice through the ring, producing many mistakes.

Task A1: *Describe the boundary you see and explain why it has trouble separating the ring in this setting.*

Next, introduce a small amount of **nonlinearity**. Add **1 hidden layer** with **6 neurons** and set **Activation** to **Tanh**, then press Play and let training run for roughly 100–200 steps. A **hidden layer** together with a **nonlinear activation function** allows the model to **bend** its decision boundary. With a sufficient number of neurons, the boundary can **wrap around** the ring and achieve a clean separation.

Task A2: *Describe how the boundary changed compared with A1. Then reduce the number of neurons from 6 to 4 and then to 2, and note the smallest neuron count that still separates the ring reasonably well. Briefly comment on what this reveals about model capacity. Provide a screenshot of your best decision boundary, which visual aspects display its quality?*

1.3 Learning Rate and Regularization

Keep the architecture from A2 and explore the effect of the **learning rate**. Set the **Learning rate** first to **0.003** and then to **1.0**. The **learning rate** controls the **step size** during training updates: a **small** learning rate makes **small, stable** steps but can be **slow** to improve; a **very large** learning rate can **overshoot**, causing **oscillations** or poor final results. Watch how the training behavior and the quality of the final boundary change.

Task B1: *Compare 0.003 and 1.0 in terms of speed, stability, and final boundary quality.*

Reset the **Learning rate** to **0.03** and turn on **Regularization = L2** with **Rate = 0.03**. **L2 regularization** gently discourages large weights, favoring **simpler, smoother** decision boundaries and often improving **robustness** to noise. Now increase the **Noise** slider from **0%** up to **30%**. With more noise, points are messier; highly flexible models are more prone to **overfitting** random patterns.

Task B2: *State whether the boundary with L2 looks visibly smoother than without it and how test performance changes. With 30% noise, which setting (L2 on or off) appears more robust, and why? Provide a screenshot of the setup and result.*

2 Image Classification (in Jupyter a Notebook)

Classification is a core task in supervised machine learning where a model learns to assign an input (e.g., an image) to one of a set of **labels**. The model is trained on examples with known labels and then evaluated on unseen data to find out how well it generalizes. Typically the **accuracy** metric is used to evaluate the classification's quality.

In the following tasks, you will complete a simple image classifier for **cats vs. dogs** using a public Kaggle dataset. The task blends in as a practical introduction to the standard workflow: you will define the goal and baseline, **split** the data into train/test sets, train **models**, and report metrics such as the **accuracy**.

Please use the Jupyter Notebook "1_Case_AEC.ipynb_TASK". Here, you will find basic code for image classification that requires completion at important parts (marked with "TODO"). For each of these, please try different configurations and observe and document their effect on the classification outcome, be able to reason as to the different outcomes:

1) Try at least two different train/ test split sizes
2) Try at least two different optimizers
3) Try at least two different metrics
4) Try different metric visualizations using a table and at least two graphs

Before you start, make sure you download the image files maintaining the specified folder structure.

3 Glossary

- **Activation function (e.g., Tanh vs. ReLU):** The activation adds **nonlinearity**, which is what lets the boundary **bend**. **Tanh** tends to produce smooth, symmetric curves around the origin, which fits the circle nicely. **ReLU** is piecewise-linear and can make boundaries that look more **polygonal**; it can still work but may look less smooth. If beginners are unsure, **Tanh** is a safe first choice here.
- **Hidden layers (model depth):** Adds extra computation between inputs and outputs so the model can draw **curved** boundaries, not just straight lines. With **0** layers the boundary is basically **linear**; adding **1** layer already lets it bend around the circle. Too **few** layers → underfitting (can't match the shape); too **many** can make training harder and increase overfitting risk. In our task, **one** hidden layer is usually enough.
- **L2 regularization (and its rate/strength):** L2 gently pushes weights toward **smaller values**, which makes the boundary **smoother** and reduces overfitting. It's most helpful when the data or labels are **noisy**. The **rate** controls how strong this push is: too **low** has little effect, too **high** can **underfit** (over-smoothed boundary). In our B-part, turning L2 **on** typically improves **robustness** at 30% noise.
- **Learning rate (LR):** Controls the **step size** of each training update. A **small** LR is **stable but slow**; the boundary improves gradually. A **large** LR can be **fast** but may **overshoot** and cause the boundary to jump or oscillate. If the plot looks chaotic, **lower** the LR; if progress is painfully slow, **raise** it slightly.
- **Neurons per hidden layer (model width):** More neurons give the layer more "building blocks" to represent **richer shapes**. With **too few** (e.g., 2) the ring often won't be separated; with **4–6** it usually works. Adding many neurons can bring **diminishing returns** and may overfit if the data are noisy. Start modestly and adjust based on what you see.
- **Noise level (data difficulty slider):** Noise makes the classification task **messier** by adding "wrong-looking" or ambiguous points. As noise increases, complex boundaries tend to **overfit** random fluctuations. L2 helps by keeping the boundary **simpler**. At high noise, don't expect perfect separation—look for **robust** behavior, not perfection.
- **Training steps / epochs:** This is simply **how long** you train. Too **few** steps and the model hasn't learned enough (underfits); too **many** steps without regularization can start to **overfit** noise. In the Playground, you will see the boundary **settle** after a while—stopping there is often fine. Use roughly **100–200** steps for quick comparisons.